Week 9 Level 2: Intermediate

Event-Driven Games

Keys and timers make things happen; a game loop moves the world one frame at a time.

What you will learn

- How to make keys call your functions with `onkey` and `listen`.
- How a **game loop** with `ontimer` moves everything one frame at a time.
- How to keep the game's **state** in a dictionary and check if two things are **touching**.
- How to start planning your own final project.

The big idea: games wait for events

Most programs you wrote ran from top to bottom and stopped. A game **waits**. When something happens (a key press, a timer ringing), turtle calls one of your functions. The thing that happens is an **event**; the function that answers it is a **handler**.

Keys call handlers

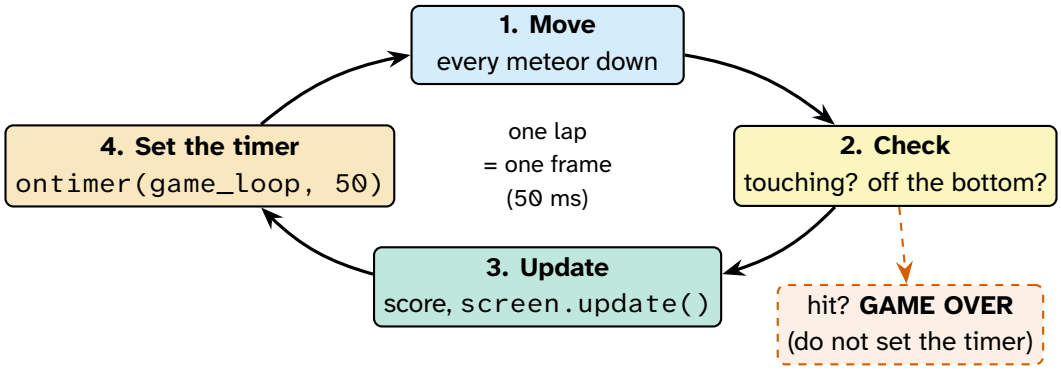
```
def go_left():
    player.setx(player.xcor() - 20)

screen.onkey(go_left, "Left")   # when Left is pressed, call go_left
screen.listen()                 # switch on the "doorbell" for keys
```

Notice: `go_left` has **no brackets** in the `onkey` line. We hand turtle the function, and turtle calls it later. With brackets, Python would call it right away, only once.

The game loop

A game is like a flip book: every **frame** the world moves a tiny bit. Our game loop does one frame, then asks turtle to call it again soon.



One frame of Dodge the Meteors (shortened)

```
state = {"score": 0, "running": True}

def game_loop():
    for meteor in meteors:                      # a list of turtles
        meteor.sety(meteor.ycor() - 8)          # 1. move
        if is_touching(meteor.xcor(), meteor.ycor(),
                       player.xcor(), player.ycor(), 30):
            state["running"] = False            # 2. check: hit!
    if state["running"]:
        screen.ontimer(game_loop, 50)           # 4. again in 50 ms
```

Touching = close enough

Two things touch when the distance between them is small. `math.dist` measures the distance between two points: `math.dist((0, 0), (30, 40))` gives `50.0`. If that is less than about 30, the meteor hit the player.

Exercise 9.1: Etch-a-Sketch

Folder: `exercises/week09/ex-etch-a-sketch`. Run `starter.py`, **click the window**, press the Up arrow.

1. TODO 1: add `screen.listen()`. Now Up draws a line.
2. TODO 2: write `go_down()`, `go_left()`, `go_right()` (headings 270, 180, 0).
3. TODO 3: connect the keys: `"Down"`, `"Left"`, `"Right"`, `"space"`, `"n"`, `"c"`.
4. TODO 4: `c` clears the drawing. Draw something and describe it here:

Exercise 9.2: Dodge the Meteors

Folder: `exercises/week09/ex-dodge-meteors`. You are the blue turtle; the round orange-red meteors fall.

1. TODO 1: `move_player()` returns the new x, using `clamp()` so you stop at the walls.
2. TODO 2: `is_touching()` uses `math.dist` and <.
3. TODO 3: connect `"Left"` and `"Right"`, then `listen()`.
4. TODO 4: at the end of `game_loop()`, set the timer again. Now play! My best score: _____
5. Which functions are **logic** (no turtle) and which are **drawing**? Why is it useful to keep them apart?

Your final project starts now!

For the next three weeks you will build your own program, alone or with a partner. Read the **Project Brief** and choose:

Option A: Arcade Game	Option B: Art Studio
A turtle game with keys, a timer and a score, like Dodge the Meteors or Star Catcher. The file saves high scores.	A program that makes designs (often random) and can save and load them, like Dot Art from a File.

Both must use: functions that return values, a list, a dictionary, a loop or game loop, a file, and at least 3 tests. **Milestone M1 (start of next week):** Planning page 1 (pitch), page 2 (pseudocode or flowchart) and, in a pair, page 3 (task split).

Tip: Borrowing pictures or sounds

Pictures and sounds belong to the people who made them. Only use ones with an open license (CC0: free for anyone; CC BY: free if you give credit), ask a grown-up to help you find them, and write a credit in your code, like `# Star sound by Mei (CC BY)`. Turtle's own shapes are always fine!

Vocabulary box

event Something that happens while a program runs: a key press, a click, a timer ringing.

handler The function that runs when an event happens.

listen() Tells the turtle window to start hearing key presses.

game loop A function that does one frame of the game, then sets a timer to run again.

frame One tiny step of a game: everything moves a little, then it is drawn.

collision Two things touching. We check it with the distance between them.

license / attribution The rules for using someone's work / giving them credit for it.

🖌 Reflection

What happens if you forget `ontimer` at the end of the game loop?

Explain to a friend why `onkey(go_left(), "Left")` does not work.

My project idea in one sentence:

Week 9 Level 2: Intermediate

Event-Driven Games

15–20 minutes. Do both Core parts; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Part 1: Stamp Studio (about 10 minutes)

Folder: `exercises/week09/hw-stamp-studio`. An art program you play with the keyboard. The current shape and color are kept in a dictionary called `studio`.

1. Open `starter.py`, press **Run** and **click the window**. Arrows move, `s` stamps. `p` and `n` do nothing yet.
2. TODO 1: `next_index(index, length)` returns the next position and goes back to `0` after the last one. Fill in: `next_index(2, 4)` = _____ `next_index(3, 4)` = _____
3. TODO 2: write `next_color()`, a twin of `next_shape()`.
4. TODO 3: connect `"p"` and `"n"` with `screen.onkey(...)`. No brackets after the function name!
5. Make a picture, then run `check_my_work.py`.

★ Core (everyone): Part 2: Project pitch, milestone M1 (about 8 minutes)

Take out your **Project Brief**. Fill in **Planning page 1: My pitch**. If you work with a partner, do it together (in person, or bring your ideas next week). Bring the Brief to the next session: we start by checking milestone M1.

- I chose: ☐ Option A: Arcade Game ☐ Option B: Art Studio
- The keys or buttons my program will use: _____

★★ Challenge (want more?): Plan it, upgrade it

- Start **Planning page 2** of the Project Brief: write the pseudocode or draw the flowchart of your project's main loop.
- Or add a key to Stamp Studio that makes the stamps bigger and smaller with `stamper.shapesize(...)`. Keep the size in the `studio` dictionary.

Creative (make it yours): Draw your game screen

On paper, draw what your project will look like when it is running. Label every part: the player or brush, the things that move, the score or menu, and the keys. Use shapes and labels, not only colors, so everyone can tell the parts apart.

Self-check: tick each box when it is true

- ☐ In Stamp Studio, p and n go round all the shapes and colors and start again.
- ☐ I wrote `onkey(next_shape, "p")` with no brackets after `next_shape`.
- ☐ My pitch page is filled in, and I know which option I chose.
- ☐ If I want to use a picture or sound, I know it needs an open license and a credit.